

Section 1: Comprehensive Foundations & First Principles of Deepfake Detection

Table of Contents

1. Introduction & The Modern Threat Landscape
2. The Physics & Mathematics of Digital Images & Video
3. 2.1 Photons to Photosensors (Hardware Capture)
4. 2.2 Digital Representations & Color Channels
5. 2.3 Tensor Dimensions in PyTorch (2D, 3D, 4D, and 5D)
6. 2.4 Statistical Normalization & Numerical Stability
7. How Deepfakes Are Synthesized (Algorithmic Mechanisms)
8. 3.1 The 5 Canonical Manipulation Technologies
9. 3.2 The Autoencoder Face-Swapping Pipeline
10. 3.3 Blending, Warping & Boundary Feathering
11. The Forensic Flaws: Why Deepfakes Break Physical Laws
12. 4.1 Photo-Response Non-Uniformity (PRNU) & Sensor Noise Disruption
13. 4.2 Transposed Convolutions & Checkerboard Frequency Artifacts
14. 4.3 Boundary Step-Discontinuities & Resolution Mismatch
15. Why Standard Deep Learning Models Fail
16. 5.1 The Semantic Bias of Standard CNNs and Vision Transformers
17. 5.2 Shortcut Learning & Identity Leakage
18. 5.3 Downsampling & High-Frequency Attenuation
19. Spatial vs. Frequency Domains: A Mathematical Primer
20. 6.1 Intuition: Coordinates vs. Sinusoidal Oscillations
21. 6.2 Complex Numbers & Euler's Formula
22. 6.3 The 2D Discrete Fourier Transform (2D DFT) Equation
23. 6.4 Magnitude Spectrum vs. Phase Spectrum
24. 6.5 The Role of Quadrant Shifting (`fftshift`)
25. Steganographic Rich Models (SRM) & High-Pass Filtering
26. 7.1 What is Steganography & Steganalysis?
27. 7.2 Mathematical Construction of SRM Filter Kernels
28. 7.3 Proof of Low-Frequency Content Cancellation
29. Adaptive Forgery Detection: Bayar-Stamm Constrained Convolutions
30. 8.1 Prediction Error Filters

- 31. [8.2 Mathematical Constraint Formulation](#)
 - 32. [Symmetric Gated Residual Fusion: Mathematical Principles](#)
 - 33. [9.1 The Problem of Gradient Starvation](#)
 - 34. [9.2 Mathematical Formulation of Dual-Stream Gating](#)
 - 35. [Probability Calibration & Classification Metrics](#)
 - [10.1 Logits, Sigmoid, and Overconfidence](#)
 - [10.2 Temperature Scaling via SciPy L-BFGS-B](#)
 - [10.3 Expected Calibration Error \(ECE\) Formulation](#)
 - [10.4 Metrics: Precision, Recall, F1-Score, ROC Curves, and AUC](#)
 - 36. [Project Blueprint & Architecture Mapping](#)
-

1. Introduction & The Modern Threat Landscape

Digital face manipulation—commonly referred to as **Deepfake** technology—has evolved from an academic curiosity into a pervasive technological challenge. Using modern deep generative architectures (Generative Adversarial Networks [GANs], Variational Autoencoders [VAEs], and Diffusion Models), bad actors can synthesize realistic video footage of individuals saying or doing things they never did.

The core objective of this project is to build an **academic-grade, production-ready, highly generalizable deepfake detection engine** that does not simply memorize facial identities, but rather detects the fundamental physical and mathematical inconsistencies introduced during neural video synthesis.

2. The Physics & Mathematics of Digital Images & Video

To understand forensic forgery detection, we must begin with how a digital camera captures visual reality and encodes it into computer memory.

2.1 Photons to Photosensors (Hardware Capture)

When light passes through a camera lens, it strikes an active-pixel sensor (such as a CMOS or CCD sensor).

1. The sensor is a two-dimensional grid of light-sensitive photodiode elements.
2. Photons hitting a photodiode are converted into an electric charge proportional to the light intensity.
3. An analog-to-digital converter (ADC) quantizes this continuous voltage into a discrete numeric value.
4. A **Bayer Color Filter Array (CFA)** places alternating Red, Green, and Blue optical filters over the sensor grid so that individual pixels measure specific wavelengths of light.
5. In addition to the true scene light, hardware imperfections in each individual silicon photodiode introduce a subtle, spatially deterministic noise pattern termed **Photo-Response Non-Uniformity (PRNU)**.

2.2 Digital Representations & Color Channels

A standard digital color image is mathematically represented as a three-dimensional array (tensor) of real or integer numbers:

$$I \in \mathbb{R}^{3 \times H \times W} \quad \text{or} \quad I \in [0, 255]^{3 \times H \times W}$$

where: - H is the image height (number of pixel rows). - W is the image width (number of pixel columns). - 3 denotes the RGB color channels: - **Red Channel (R)**: Intensity of red light at coordinate (x, y) . - **Green Channel (G)**: Intensity of green light at coordinate (x, y) . - **Blue Channel (B)**: Intensity of blue light at coordinate (x, y) .

2.3 Tensor Dimensions in PyTorch (2D, 3D, 4D, and 5D)

Throughout this codebase, tensors transition through different dimensional geometries:

Tensor Rank	Shape Geometry	Forensic Meaning & Usage
2D Tensor	[H, W]	Grayscale image, Grad-CAM heatmap, single FFT spec
3D Tensor	[C, H, W]	Single color face crop (C=3 RGB channels)
4D Tensor	[B, C, H, W]	Batch of face crops (B=Batch Size, e.g., 16 images)
5D Tensor	[B, T, C, H, W]	Batch of temporal video sequences (T=Sequence Length)

2.4 Statistical Normalization & Numerical Stability

Raw 8-bit image pixels exist in the range $[0, 255]$. Passing large integer values into neural networks causes gradient explosion and unstable activation distributions.

In this codebase, images are converted to 32-bit floating-point numbers in $[0.0, 1.0]$ and standardized using **ImageNet distribution parameters**:

$$I_{\text{norm}}(c, x, y) = \frac{\frac{I(c, x, y)}{255.0} - \mu_c}{\sigma_c}$$

where:

$$\mu = [0.485, 0.456, 0.406], \quad \sigma = [0.229, 0.224, 0.225]$$

This guarantees that spatial input features have zero mean ($\mathbb{E}[I] \approx 0$) and unit variance ($\text{Var}(I) \approx 1$), providing numerical stability for gradient descent.

3. How Deepfakes Are Synthesized (Algorithmic Mechanisms)

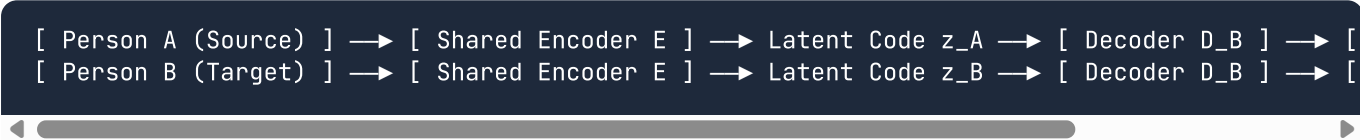
3.1 The 5 Canonical Manipulation Technologies

This project is engineered and benchmarked against the two definitive forensic benchmarks: **FaceForensics++ (FF++)** and **Celeb-DF v2**. These datasets cover 5 distinct generative methods:

Manipulation Method	Underlying Algorithm	Forensic Signature Left Behind
1. FF++ Deepfakes	Dual-Autoencoder (DeepFaceLab)	Blending boundaries, low-res skin
2. FF++ Face2Face	3D Morphable Model (3DMM)	Mouth/eye warping, photometric se
3. FF++ FaceSwap	Classical 3D Graphics Mesh	Texture stitching seam along fac
4. FF++ NeuralTextures	Patch-based Neural Rendering	Loss of high-frequency micro-pore
5. Celeb-DF v2	High-Res Deep Generative Model	High quality; subtle 2D FFT anom

3.2 The Autoencoder Face-Swapping Pipeline

The most widespread deepfake creation pipeline utilizes a **Shared Encoder + Split Decoder** architecture:



- Shared Encoder $E(x)$:** Trained on both Person A and Person B to map facial geometry, expressions, and head pose into an identity-agnostic latent space $\mathbf{z} \in \mathbb{R}^d$.
- Target Decoder $D_B(\mathbf{z})$:** Specifically trained to reconstruct the facial appearance of Person B.
- Inference (Face Swap):** A frame of Person A is passed through E , producing latent code $\mathbf{z}_A$. This vector is routed to D_B , which reconstructs Person B making Person A's exact expression.

3.3 Blending, Warping & Boundary Feathering

The generated face from D_B is only a square crop (e.g., 128×128 or 256×256). To insert it into the target video frame: 1. **Affine Landmark Alignment:** An inverse affine matrix warps the synthetic face back to the target's head pose. 2. **Color Correction:** Histogram matching or linear color transfer adjusts skin illumination. 3. **Alpha Feathering / Poisson Blending:** A soft binary mask $M \in [0, 1]$ blends the synthetic face I_{synth} with the original background I_{orig} :

$$I_{\text{composite}} = M \odot I_{\text{synth}} + (1 - M) \odot I_{\text{orig}}$$

4. The Forensic Flaws: Why Deepfakes Break Physical Laws

When a deep generative model synthesizes and pastes a fake face, it inadvertently violates the underlying physics of digital photography in three major ways.

4.1 Photo-Response Non-Uniformity (PRNU) & Sensor Noise Disruption

- **Authentic Photos:** In a genuine video frame, the microscopic high-frequency sensor noise $n_{\text{sensor}}(x, y)$ is physically continuous across the entire frame (from forehead to cheek to background).
- **Deepfake Forgery:** The neural network reconstructs pixels using smooth mathematical activation functions (ReLU, GELU, Sigmoid). It **cannot reproduce the physical camera's PRNU noise**.
- **The Forensic Result:** The inner facial mask has smooth, synthetic noise, whereas the outer frame contains genuine camera sensor noise. The boundary where the face was pasted exhibits an invisible **noise step-discontinuity**.

4.2 Transposed Convolutions & Checkerboard Frequency Artifacts

To scale low-dimensional latent vectors up to high-resolution pixel matrices, neural networks use **transposed convolutions** (`ConvTranspose2d`) or **sub-pixel convolution layers**.

```
Kernel with size k=3, stride s=2:
Input:      [ a ]      [ b ]
            / | \      / | \
Overlap Grid: [ .  x  X  x  . ] ← Where adjacent receptive fields overlap,
                                   pixel values receive double accumulation!
```

This uneven overlap creates a periodic, high-frequency spatial variation known as the **checkerboard artifact**. While barely visible to the human eye, in the **Fourier frequency spectrum** these periodic patterns manifest as distinct, unnatural geometric peaks and spikes.

4.3 Boundary Step-Discontinuities & Resolution Mismatch

Generative networks often output face crops at fixed lower resolutions (e.g., 256×256) that are subsequently upscaled and blended into high-definition (1080p) video frames. - This creates an abrupt **resolution mismatch**: high-frequency skin pores and hair strands exist outside the mask, but are blurred or missing inside the synthetic facial region. - Smoothing algorithms (e.g., Gaussian blur on the mask edge) create an unnaturally smooth transition zone around the jawline and forehead.

5. Why Standard Deep Learning Models Fail

5.1 The Semantic Bias of Standard CNNs and Vision Transformers

Standard vision architectures (ResNet, VGG, ViT, EfficientNet) were designed for **object categorization** (e.g., ImageNet classification). - They are trained to achieve **invariance** to noise and texture variations so they can focus on high-level semantic concepts (e.g., "Is there an eye? Is there a mouth?"). - In a deepfake, the semantic layout is completely intact: the face has two eyes, a nose, and a mouth in anatomically correct positions. - Because standard networks ignore subtle high-frequency residuals, they are easily deceived.

5.2 Shortcut Learning & Identity Leakage

If a dataset contains 50 real videos of Actor X and 50 fake videos of Actor Y, a naive neural network will learn to recognize the facial features of Actor Y rather than detecting forgery artifacts. - When tested on an unseen Actor Z, the model's accuracy drops to near random guessing (50%). - Preventing this requires

strict identity-disjoint graph partitioning where no actor identity ever appears in both training and test sets.

5.3 Downsampling & High-Frequency Attenuation

Standard CNN architectures rapidly reduce spatial dimensions via early strided convolutions ($s = 2$) and max-pooling operations (2×2). - Mathematically, pooling acts as a **low-pass filter**, averaging adjacent pixels and wiping out the exact micro-noise residuals where forensic traces reside.

6. Spatial vs. Frequency Domains: A Mathematical Primer

6.1 Intuition: Coordinates vs. Sinusoidal Oscillations

[Spatial Domain: $f(x, y)$]	[Frequency Domain: $F(u, v)$]
"Where are features located in space?"	"How rapidly do color values oscillate?"
• x : Horizontal pixel coordinate (0 to $W-1$)	• u : Horizontal spatial frequency (cycles per pixel)
• y : Vertical pixel coordinate (0 to $H-1$)	• v : Vertical spatial frequency (cycles per pixel)
• Value: RGB color intensity at (x, y)	• Value: Amplitude and phase of wave

- **Low Frequencies (Center of Spectrum)**: Smooth skin, flat lighting, broad facial contours.
- **High Frequencies (Outer Edges)**: Sharp edges, fine hair, microscopic noise, artificial upsampling checkerboards.

6.2 Complex Numbers & Euler's Formula

Fourier analysis represents waves using complex numbers. According to Euler's identity:

$$e^{j\theta} = \cos(\theta) + j \sin(\theta), \quad \text{where } j = \sqrt{-1}$$

A complex number $z = a + jb$ can be plotted on a 2D plane: - Real component: $a = \text{Re}(z) = |z| \cos(\theta)$ - Imaginary component: $b = \text{Im}(z) = |z| \sin(\theta)$ - Magnitude: $|z| = \sqrt{a^2 + b^2}$ - Phase Angle: $\theta = \text{atan2}(b, a)$

6.3 The 2D Discrete Fourier Transform (2D DFT) Equation

For a 2D digital image matrix $f(x, y)$ of height H and width W , the 2D Discrete Fourier Transform decomposes the image into a sum of complex sinusoidal basis functions:

$$F(u, v) = \sum_{x=0}^{H-1} \sum_{y=0}^{W-1} f(x, y) \cdot e^{-j2\pi\left(\frac{ux}{H} + \frac{vy}{W}\right)}$$

where: - $u \in [0, H - 1]$ is the horizontal frequency index. - $v \in [0, W - 1]$ is the vertical frequency index. - $F(u, v) \in \mathbb{C}$ is a complex value holding the amplitude and phase of that specific 2D spatial frequency.

6.4 Magnitude Spectrum vs. Phase Spectrum

From each complex frequency coefficient $F(u, v) = R(u, v) + jI(u, v)$, we extract two distinct forensic representations:

1. **Magnitude Spectrum** $|F(u, v)|$: Measures the total energy (strength) present at frequency (u, v) :

$$|F(u, v)| = \sqrt{R(u, v)^2 + I(u, v)^2}$$

Because the zero-frequency (DC) component is orders of magnitude larger than high-frequency noise, we apply a **logarithmic dynamic range compression**:

$$\mathcal{M}(u, v) = \ln (|F(u, v)| + 1)$$

2. **Phase Spectrum** $\Phi(u, v)$: Encodes the spatial alignment, structural edges, and phase coherence of the image:

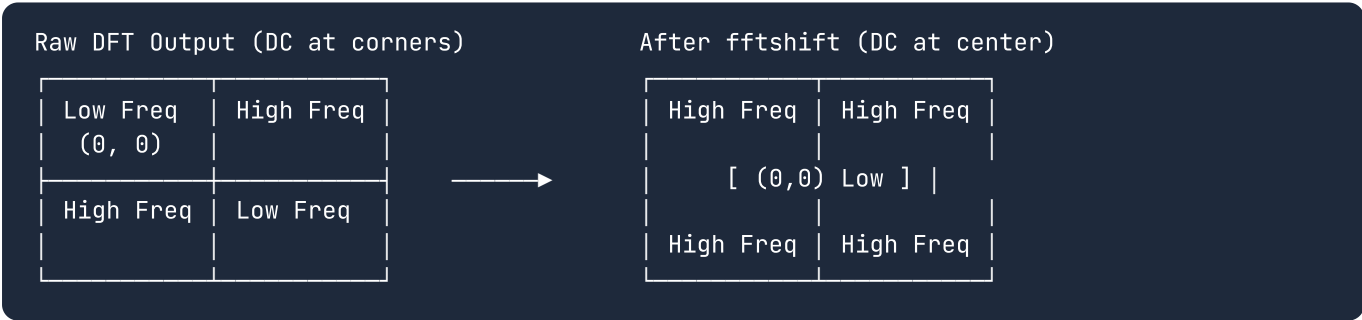
$$\Phi(u, v) = \text{atan2}(I(u, v), R(u, v)) \in [-\pi, +\pi]$$

Normalized to $[-1.0, +1.0]$ for neural network consumption:

$$\Phi_{\text{norm}}(u, v) = \frac{\Phi(u, v)}{\pi}$$

6.5 The Role of Quadrant Shifting (`fftshift`)

In standard FFT implementations, the zero-frequency component ($u = 0, v = 0$, representing average image brightness) originates in the top-left corner. - `fftshift` performs a circular shift that swaps diagonal quadrants (1st with 3rd, 2nd with 4th). - This shifts $(0, 0)$ to the **exact center** of the spectral matrix:
- **Center**: Low frequencies (macro structures). - **Periphery**: High frequencies (micro noise and artifacts).



7. Steganographic Rich Models (SRM) & High-Pass Filtering

7.1 What is Steganography & Steganalysis?

- **Steganography**: The practice of concealing secret information inside digital images by making microscopic, visually imperceptible modifications to pixel values.
- **Steganalysis**: The science of detecting these hidden modifications.

- In 2012, digital forensics pioneers (Fridrich & Kodovsky) established the **Steganographic Rich Model (SRM)**: a bank of hand-engineered high-pass spatial filters designed to suppress the image's scene content and isolate the underlying **noise residuals**.

7.2 Mathematical Construction of SRM Filter Kernels

In `src/models/hybrid_detector.py#L25-L36`, three specialized 5×5 SRM kernels are hardcoded:

SRM 1st-Order Kernel	SRM 2nd-Order Kernel	SRM Edge / Square
$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 \\ 0 & 2 & -4 & 2 & 0 \\ 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} / 4$	$\begin{bmatrix} -1 & 2 & -2 & 2 & -1 \\ 2 & -6 & 8 & -6 & 2 \\ -2 & 8 & -12 & 8 & -2 \\ 2 & -6 & 8 & -6 & 2 \\ -1 & 2 & -2 & 2 & -1 \end{bmatrix} / 12$	$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & -2 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$

7.3 Proof of Low-Frequency Content Cancellation

Notice that for every SRM kernel K , the sum of all coefficients is exactly zero:

$$\sum_{i=-2}^2 \sum_{j=-2}^2 K(i, j) = 0$$

Mathematical Proof of Image Suppression: Consider an image region where skin tone is smooth and uniform with constant intensity C :

$$I(x + i, y + j) = C \quad \forall i, j \in -2, \dots, 2$$

The 2D convolution output $R(x, y)$ at that location is:

$$R(x, y) = \sum_{i=-2}^2 \sum_{j=-2}^2 I(x + i, y + j) \cdot K(i, j) = \sum_{i=-2}^2 \sum_{j=-2}^2 C \cdot K(i, j) = C \cdot \sum_{i=-2}^2 \sum_{j=-2}^2 K(i, j) = C \cdot 0 = 0$$

- **Result:** Homogeneous facial regions (cheeks, forehead) are completely extinguished to **zero (black)**.
- Only regions with artificial pixel discontinuities, abnormal noise variances, or boundary seams produce non-zero residual responses.

8. Adaptive Forgery Detection: Bayar-Stamm Constrained Convolutions

While SRM kernels are fixed, generative models may produce subtle artifacts that static filters miss. In 2016, Bayar and Stamm developed a **constrained convolutional layer** that adaptively learns forgery-specific

prediction error filters during backpropagation.

8.1 Prediction Error Filters

A prediction error filter estimates the central pixel from its surrounding neighborhood and subtracts the estimate from the actual central pixel:

$$R(x, y) = I(x, y) - \hat{I}(x, y) = I(x, y) - \sum_{(i,j) \neq (0,0)} w_{i,j} I(x-i, y-j)$$

8.2 Mathematical Constraint Formulation

In `src/models/hybrid_detector.py#L58-L71`, the learnable weights w of a 5×5 kernel are constrained on every forward pass:

$$\left\{ \begin{aligned} w_{\text{center}} &= w(0, 0) = -1.0 \\ \sum_{(i,j) \neq (0,0)} w(i, j) &= 1.0 \end{aligned} \right.$$

Algorithm in Code: 1. Zero out the center weight: $w_{\text{masked}} = w \odot (1 - \delta_{0,0})$. 2. Compute sum of non-center weights: $S = \sum_{(i,j) \neq (0,0)} w(i, j)$. 3. Normalize non-center weights by S : $w_{\text{norm}} = \frac{w_{\text{masked}}}{S}$. 4. Set the center coefficient to -1.0 :

$$w_{\text{constrained}} = w_{\text{norm}} - \delta_{0,0}$$

This mathematical constraint forces the kernel to **learn only prediction error residuals**, preventing it from collapsing into a standard semantic feature extractor.

9. Symmetric Gated Residual Fusion: Mathematical Principles

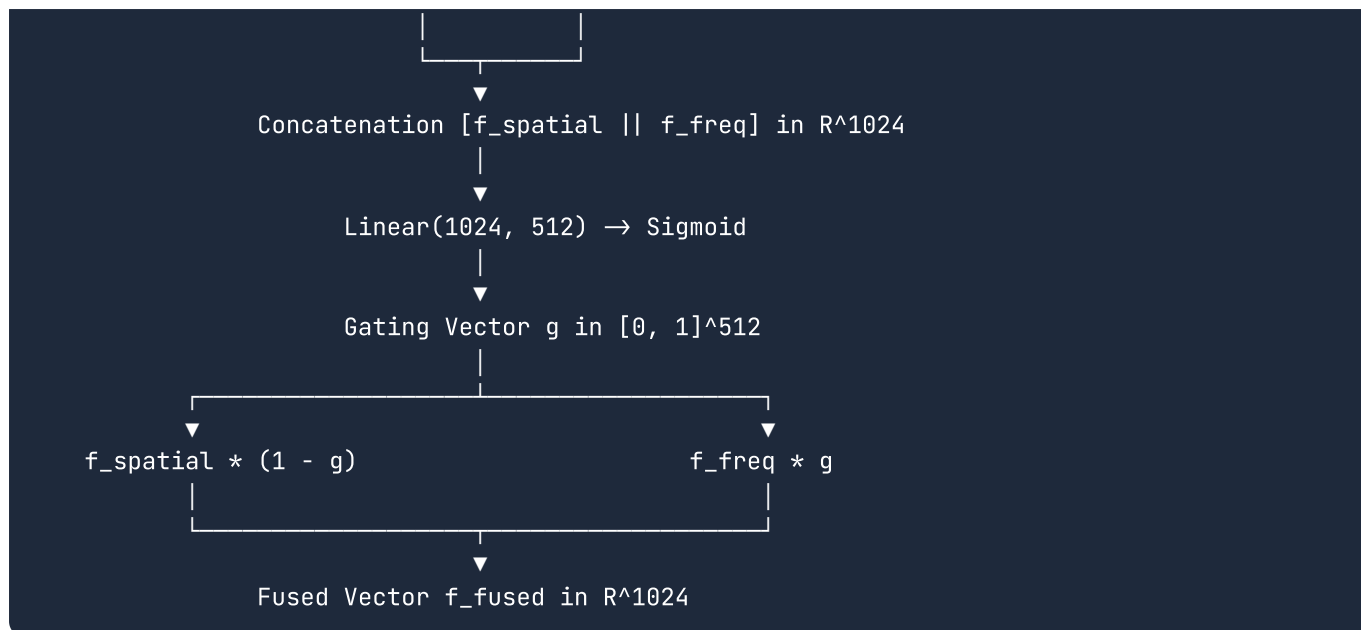
9.1 The Problem of Gradient Starvation

In multi-modal or dual-stream neural networks, one branch often trains faster than the other. - The spatial backbone (ConvNeXt) is initialized with ImageNet pre-trained weights, whereas the frequency branch is trained from scratch. - In a naive concatenation ($[\mathbf{f}_{\text{spatial}} \parallel \mathbf{f}_{\text{freq}}]$), the optimizer takes the path of least resistance, quickly driving spatial weights to minimize loss while ignoring the frequency stream. - The frequency stream suffers from **gradient starvation** and never learns meaningful spectral representations.

9.2 Mathematical Formulation of Dual-Stream Gating

To eliminate gradient starvation, this project implements **Symmetric Gated Residual Fusion** in `src/models/hybrid_detector.py#L189-L194`:

```
Spatial Stream Vector:  f_spatial in R^512
Frequency Stream Vector: f_freq    in R^512
```



1. **Gating Vector Computation:** $g = \sigma(\mathbf{W}g + \mathbf{f}_{\text{spatial}} + \mathbf{f}_{\text{freq}} + \mathbf{b}_g) \in [0, 1]^{512}$
2. **Symmetric Modulation:** $\mathbf{f}_{\text{fused}} = [\mathbf{f}_{\text{spatial}} \odot (1 - g) \parallel \mathbf{f}_{\text{freq}} \odot g] \in \mathbb{R}^{1024}$

Why is this mathematically symmetric? - Because **both** streams are multiplied by a gating factor (g or $1 - g$), neither stream has an unconstrained shortcut to the loss function. - If the model tries to set $g \rightarrow 0$ to rely solely on spatial features, the gradient $\frac{\partial \mathcal{L}}{\partial g}$ creates backpropagation pressure that forces the gating layer to seek complementary frequency cues.

10. Probability Calibration & Classification Metrics

10.1 Logits, Sigmoid, and Overconfidence

The classifier head outputs a raw scalar $z \in (-\infty, +\infty)$ called the **logit**. - To convert this logit into a probability $p \in [0, 1]$:

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

- **The Overconfidence Pathology:** Modern deep neural networks trained with Cross-Entropy loss are notoriously overconfident: a model might output $p = 0.999$ for an ambiguous face that it only classifies correctly 70% of the time.

10.2 Temperature Scaling via SciPy L-BFGS-B

To produce trustworthy probabilities, we apply **Post-Hoc Temperature Scaling** (`src/utils/checkpoint.py#L34-L49`). - A single positive temperature parameter $T > 0$ scales the logits: $p_{\text{calibrated}} = \sigma\left(\frac{z}{T}\right)$ - To prevent $T \leq 0$ during optimization, we

reparameterize $T = \exp(\theta)$ and find the optimal θ by minimizing the **Negative Log-Likelihood (NLL)** on validation set logits using the **L-BFGS-B** algorithm:

$$\min_{\theta} -\frac{1}{N} \sum_{i=1}^N \left[y_i \ln \sigma(z_i e^{-\theta}) + (1 - y_i) \ln (1 - \sigma(z_i e^{-\theta})) \right]$$

- In this project, the optimal temperature was empirically determined to be $T^* = 1.4788$.

10.3 Expected Calibration Error (ECE) Formulation

To measure calibration quality, predictions are partitioned into $M = 15$ confidence bins $B_1, \dots, B_M$.

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{N} \left| \text{acc}(B_m) - \text{conf}(B_m) \right|$$

- **Raw Uncalibrated ECE:** 0.0122 - **Calibrated Model ECE ($T^* = 1.4788$):** 0.0093 (a 23.8% reduction in calibration error).

10.4 Metrics: Precision, Recall, F1-Score, ROC Curves, and AUC

In binary deepfake classification: - Label 0 = **Real (Authentic)** - Label 1 = **Fake (Synthesized / Manipulated)**

		Actual Condition	
		Real (0)	Fake (1)
Predicted Real (0)		[True Neg (TN)]	[False Neg (FN)]
Predicted Fake (1)		[False Pos (FP)]	[True Pos (TP)]

1. Recall (Sensitivity / Detection Rate):

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Our Model: 99.79% (catches almost all fakes).

2. Precision (Positive Predictive Value):

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Our Model: 96.86% .

3. F1-Score (Harmonic Mean):

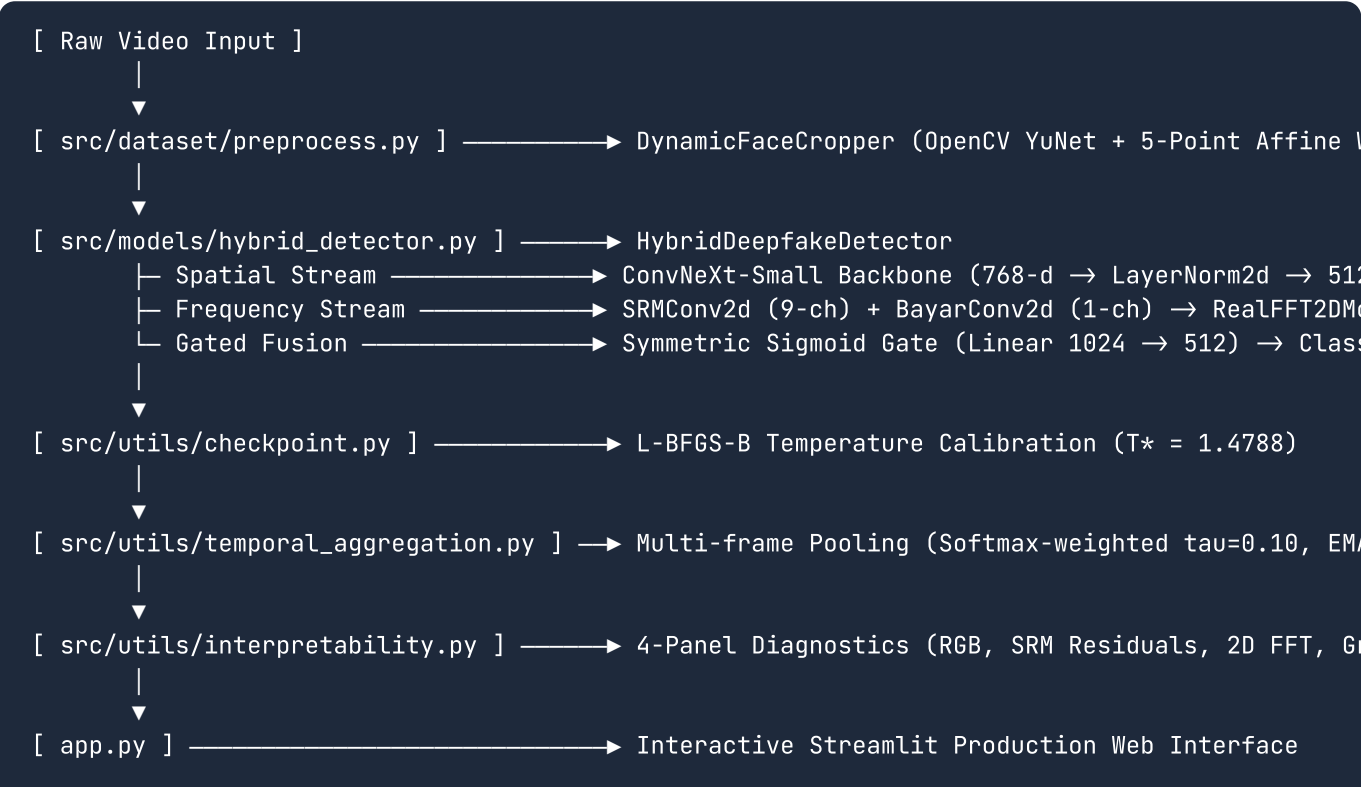
$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = 0.9830$$

4. Receiver Operating Characteristic (ROC) & Area Under Curve (AUC):

- 5. Plots True Positive Rate ($TPR = \frac{TP}{TP+FN}$) against False Positive Rate ($FPR = \frac{FP}{FP+TN}$) across all possible decision thresholds $\tau \in [0, 1]$.
- 6. **AUC = 1.0**: Perfect classifier.
- 7. **AUC = 0.5**: Random guessing.
- 8. **Our Model**: **0.9988 AUC** on 10,528 held-out test face crops.

11. Project Blueprint & Architecture Mapping

Here is how all mathematical concepts map directly to the codebase:



This document serves as the permanent theoretical foundation for Section 1 of the Deepfake Detection Engine.